

1. Introducción y objetivos

1.1. Presentación

El proyecto GymTrack consiste en el diseño y desarrollo integral de una aplicación móvil, en especial para dispositivos Android, orientada a la gestión deportiva.

El sistema se compone de una aplicación móvil cliente nativa para Android (Front-End) y un servidor centralizado (Back-End) desarrollado a través del framework Spring Boot.

El objetivo principal de este proyecto es digitalizar y optimizar la gestión de clientes, registrando sus rutinas de entrenamiento, llevando un control exhaustivo de sus parámetros de salud y permitiendo un seguimiento altamente personalizado de la progresión física de cada usuario. Toda esta información se procesa y almacena de forma segura en una base de datos relacional MySQL.

Las características principales del sistema permiten:

- Administración diferenciada y segura basada en roles de usuario (Cliente y Entrenador).
- Creación y gestión de perfiles biométricos con datos fundamentales (edad, peso, altura, neat).
- Registro y control del historial de entrenamientos y evolución de los biomarcadores de los clientes.
- Monitorización del progreso mediante métricas y estadísticas basadas en datos reales.
- Asistentes de inteligencia artificial personalizados para cada usuario.

1.2. Justificación

En la actualidad, el sector del entrenamiento de hipertrofia, y en popular el ámbito de los entrenadores personales, presenta un déficit de herramientas digitales accesibles para la gestión técnica de sus clientes. Una gran mayoría de estos profesionales sigue dependiendo de metodologías analógicas (hojas de papel) o herramientas ofimáticas genéricas (como hojas de cálculos excel).

Bajo mi punto de vista, este enfoque resulta altamente ineficiente, poco escalable y propenso a la pérdida de información, lo que dificulta ofrecer un asesoramiento profesional y basado en datos a largo plazo.

GymTrack nace para cubrir esta necesidad tecnológica, ofreciendo una solución digital centralizada, práctica y personalizable. Permite a los entrenadores profesionalizar su servicio mediante un seguimiento riguroso, mientras que otorga a los clientes una herramienta intuitiva para registrar sus entrenamientos y visualizar su evolución.

Desde una perspectiva puramente académica, este proyecto constituye el escenario ideal para integrar y demostrar las competencias clave adquiridas durante el Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM), destacando:

- Programación orientada a objetos avanzada.
- Diseño e implementación de interfaces gráficas (UI/UX) móviles.
- Arquitectura de software Cliente - Servidor.
- Desarrollo de APIs RESTful seguras e implementación de lógica de negocio por capas.
- Diseño, modelado y gestión de bases de datos relacionales.
- Uso de herramientas DevOps y despliegue.

1.3. Objetivos del proyecto

Objetivo general

El objetivo general de este proyecto es desarrollar un sistema de software integral, seguro y escalable.

Objetivos específicos

- Diseñar una interfaz de usuario (UI) clara, ergonómica y adaptada a la experiencia de usuario (UX) tanto para perfiles técnicos (entrenadores) como para usuarios finales (clientes).

- Diseñar y desplegar una base de datos relacional normalizada que soporte las entidades del dominio.
- Implementar un backend robusto, el cual exponga las operaciones CRUD necesarias para la gestión del sistema.
- Garantizar la seguridad de la información mediante la implementación de autenticación y autorización basada en tokens (JWT) y cifrado de contraseñas.
- Desarrollar la lógica necesaria para generales resúmenes e historiales de evaluación por cliente.
- Asegurar la integridad de la información mediante la validación de datos tanto en el lado del cliente (Android) como en el servidor (Spring boot).

1.4. Separación de roles

Dentro de la propia aplicación, los usuarios tendrán diferentes roles y en función de cual sea su rol tendrán acceso a una serie de funcionalidades diferentes. A continuación se detallan dichas funcionalidades.

- Cliente
 - Puede registrar sus propios entrenamientos en función de diferentes marcadores (grupo muscular, ejercicio, series, repeticiones, rir).
 - Puede modificar su propio perfil y datos en función de su algunos biomarcadores como el peso cambian a lo largo del tiempo.
- Entrenador
 - Puede llevar el registro de entrenamiento de todos los clientes asignados. También puede establecer comentarios/mensajes acerca de dichos entrenos.
 - Tendrá acceso a métricas basadas en la regresión lineal para medir el progreso de cada uno de sus clientes.
 - Tendrá un asistente de inteligencia artificial personalizado para cada uno de sus clientes, a la vez que uno para consultas más generales.

Ejemplo de navegación según el rol

Rol	Menú/Pantallas
Cliente	Inicio, Registro de entrenamiento, Mi perfil, Registro de salud, Cerrar sesión
Entrenador	Inicio, Listado de clientes, Asistente AI, Perfil, Cerrar Sesión

2. ANÁLISIS DEL SISTEMA

2.1. Estudio de la viabilidad

GymTrack no se concibe únicamente como un ejercicio académico, sino como un producto de software con potencial real de inserción en el mercado.

Para determinar el posible éxito y la viabilidad del proyecto, se ha realizado un análisis basado en cuatro dimensiones fundamentales.

Viabilidad comercial y de mercado

El sector del fitness está experimentando un crecimiento sostenido pero, paradójicamente, adolece de una fuerte brecha digital en la gestión de clientes (B2B).

- Público objetivo (target): entrenadores personales autónomos, preparadores físicos y, como extensión, su cartera de clientes.
- Ventaja competitiva: se ofrece una solución centralizada que sustituye a herramientas manuales o de software básico.
- Modelo de negocio propuesto (SaaS): a nivel comercial, la aplicación tiene potencial para ser monetizada mediante un modelo de "Software como servicio", ofreciendo una versión gratuita (Freemium) con límite de clientes.

Viabilidad económica

Uno de los puntos más fuertes del proyecto es su bajo coste inicial y de mantenimiento.

- Inversión en licencias (0 euros): la decisión estratégica de usar software libre y de código abierto (Open Source) reduce los costes de licencias de software a cero.
- Costes de infraestructuras (0 euros): actualmente el despliegue del backend se realiza a través de una plataforma gratuita (RailWay).
- Costes de distribución (25 dólares): el único coste añadido sería la cuota de registro como desarrollador en la Google Play Console.

Como conclusión, el recurso más demandado por el proyecto es el capital humano, haciendo que el riesgo financiero sea mínimo.

Viabilidad legal y ética

Al tratarse de una aplicación que gestiona perfiles de usuarios y sobre todo parámetros biométricos, el marco legal es crítico.

- Protección de datos (RGPD): el sistema cumple con la ley de protección de datos debido al uso de tecnologías como cifrado BCrypt, JWT y comunicaciones HTTPS.
- Propiedad intelectual: el uso de una licencia MIT para el código fuente asegura que el proyecto cumple con los estándares éticos de la comunidad de desarrolladores.

Viabilidad medioambiental y sostenida

En el contexto actual, cualquier iniciativa empresarial o desarrollo tecnológico debe de evaluar su impacto medioambiental. GymTrack, aunque sea un proyecto puramente

digital, su diseño se ha alineado con los objetivos de desarrollo sostenible (ODS):

- Filosofía paperless
- Eficiencia energética y computación en la nube.
- Mitigación de la basura electrónica.

El proyecto no solo es respetuoso con el medio ambiente por su naturaleza digital, sino que actúa de forma activa como un herramienta de modernización sostenible para los profesionales del entrenamiento.

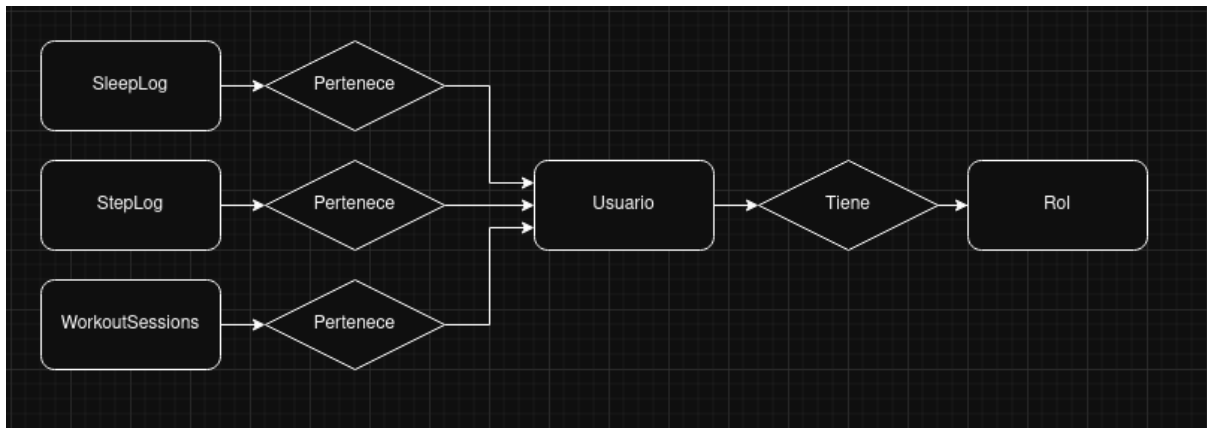
2.2. Casos de uso (entidad relación)

El análisis de los casos de uso define el comportamiento del sistema desde el punto de vista del usuario, delimitando las interacciones entre los actores involucrados y la plataforma. Para el entorno de GymTrack, se han identificado dos actores principales con diferentes niveles de privilegios y flujos de interacción: Cliente y Entrenador.

Para dar soporte a los diferentes casos de uso descritos, se ha diseñado un modelo de datos relacional, robusto y normalizado. La implementación física de este modelo reside en un motor de datos MySQL, mientras que la abstracción lógica del código fuente se gestiona mediante el estándar JPA y el framework Hibernate, permitiendo un mapeo objeto-relacional fluido.

Relaciones clave del modelo

- Cliente - Sesión de entrenamiento: 1:N
- Cliente - Log de pasos: 1:N
- Cliente - Log de sueño: 1:N
- Entrenador - Cliente: 1:N
- User - Entrenador: 1:1 (Herencia)
- User - Cliente: 1:1 (Herencia)



MODIFICAR

3. METODOLOGÍA DE TRABAJO

Este proyecto, al tratarse de un proyecto de desarrollo de software individual, la selección de la metodologías y herramientas de planificación ha sido crítica para garantizar el cumplimiento de los plazos establecidos.

Para el desarrollo de la aplicación móvil y su correspondiente infraestructura de backend, se ha optado por un ciclo de vida iterativo e incremental, complementado con prácticas ágiles adaptadas al trabajo autónomo.

Este enfoque permite dividir el proyecto en bloques funcionales o incrementos. Cada iteración abarca desde el análisis de requisitos específicos hasta su diseño, implementación y pruebas.

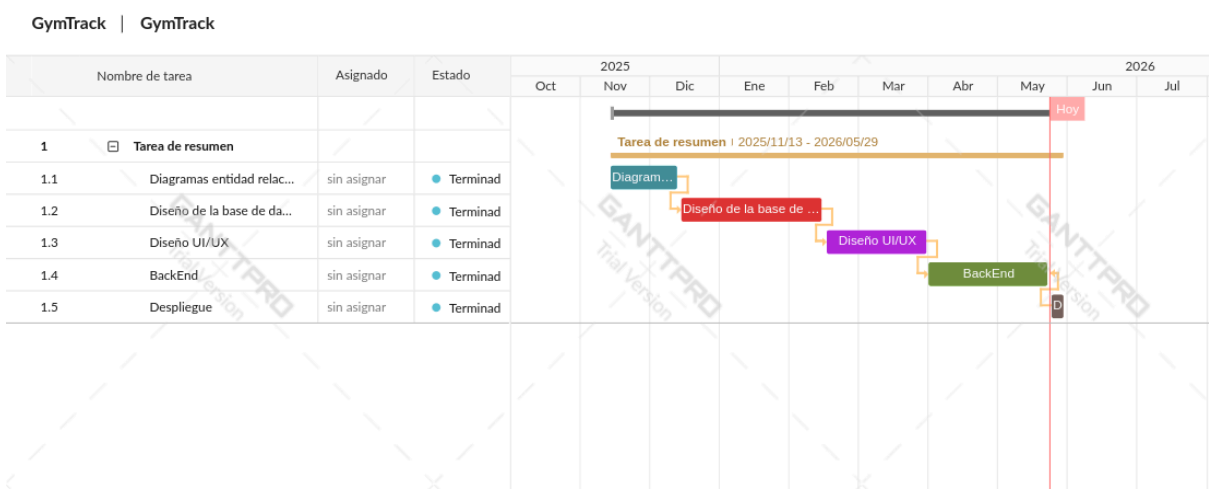
Para la planificación temporal del proyecto se ha utilizado un Diagrama de Gantt. Esta herramienta de gestión ha sido el eje vertebrador de la planificación temporal.

Gracias al diagrama de Gantt, he podido dividir el proyecto en diferentes bloques funcionales interdependientes. Estos son los siguientes:

- Diseño del diagrama de entidad relación.
- Diseño de la base de datos (posteriormente fue orquestada por JPA/Hibernate).

- Diseño de la interfaz, UI/UX.
- Diseño del backend, con Spring boot.
- Contenedorización y despliegue de la aplicación.

A continuación se muestra un diagrama de Gantt orientativo usado para el desarrollo de esta aplicación.



Gestion de configuración y control de versiones

Cabe hacer especial mención a la principal herramienta utilizada durante el desarrollo de este proyecto. El control del código fuente se gestionó mediante Git y la plataforma GitHub, aplicando una estrategia de ramificación estructurada para simular un entorno profesional y mitigar errores en entornos de exportación.

Fueron usadas multitud de ramas en este proyecto, cada una para tareas especificas, pero las tres principales son las siguientes:

- Rama "main": destinada exclusivamente al entorno de desarrollo local. Contiene configuraciones de infraestructura especificas para pruebas, que facilitan el despliegue inmediato y la corrección por parte de terceros.
- Rama "test": utilizada para la integracion de nuevas funcionalidades antes de su paso a entornos criticos.

- Rama "production": el entorno sagrado del proyecto. Esta rama contiene el código limpio de producción, parametrizado para absorber de forma transparente las variables de entorno inyectadas por el proveedor en la nube, eliminando por completo cualquier fuga de credenciales en el repositorio público.

4. DISEÑO DEL SISTEMA

4.1. Arquitectura principal del proyecto

Para el diseño y desarrollo de este proyecto, se ha establecido una arquitectura distribuida basada en el modelo Cliente - Servidor, comunicada a través de una API REST. Además, el desarrollo se fundamenta en el patrón arquitectónico Modelo - Vista - Controlador (MVC).

- El cliente (Front-End): corresponde a la aplicación móvil desarrollada para Android. Su responsabilidad es renderizar las interfaces gráficas, presentar la información al cliente y capturar sus interacciones. Este cliente delega el procesamiento de la lógica de negocio compleja realizando peticiones HTTPS (GET, POST, PUT, DELETE) al servidor.
- El servidor (Back-End): representa toda la lógica de la aplicación. El controlador implementa los diferentes endpoints de la API REST, actuando como intermediario.

Esta separación desacopla totalmente la interfaz de usuario de la lógica de datos, lo que significa que en un futuro se podría desarrollar una versión web de la aplicación o cambiar la base de datos sin que el otro extremo del sistema se vea afectado.

4.2. Diseño de la base de datos

Para la persistencia de los datos y la gestión del almacenamiento, he adoptado un enfoque basado en el paradigma Mapeo Objeto-Relacional (ORM). Esta estrategia permite tender

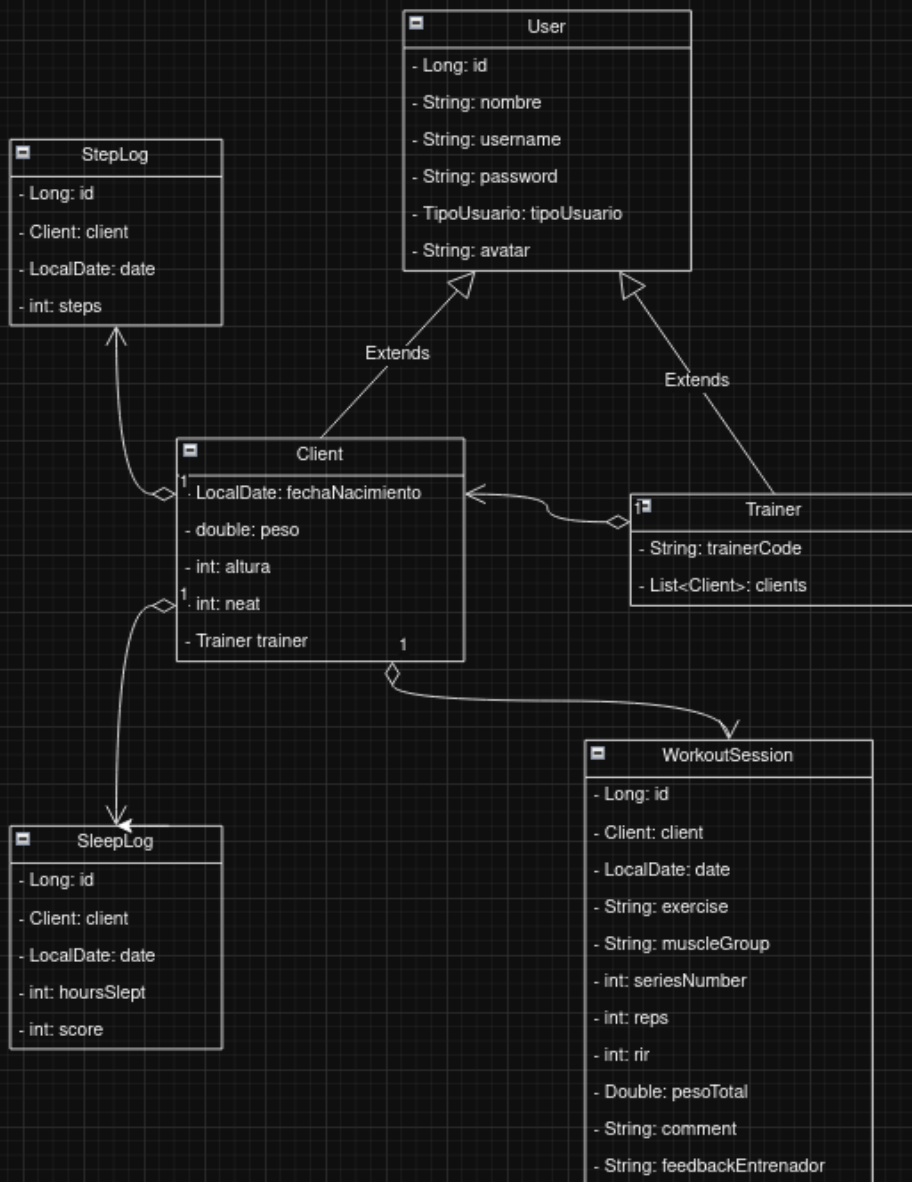
un puente entre el modelo orientado a objetos del backend y la estructura relacional de la base de datos, garantizando la integridad de los mismos y la escalabilidad del sistema.

Arquitectura de persistencia: JPA e Hibernate

La implementación de esta capa se apoya en los estándares líderes del ecosistema Java:

- JPA (Java Persistence API): es utilizada como la especificación estándar para la gestión de los datos relacionales. Al emplear JPA mediante anotaciones estructurales (@Entity, @Table, @Id, etc.), el código adquiere un alto nivel de abstracción, quedando desacoplado del proveedor de persistencia específico y asegurando una mayor portabilidad.
- Hibernate: seleccionado como el motor ORM y la implementación de JPA encargada de traducir de forma transparente las operaciones sobre los objetos Java en consultas SQL optimizadas.

Para ofrecer una visión clara y global de la arquitectura, en la siguiente figura se presenta el diagrama de clases de entidad, en el cual se detallan los atributos principales, tipos de datos y las asociaciones que definen la estructura general de la base de datos.



4.3. Diseño de la API REST

Toda la lógica de negocio de la API REST está orquestada por los controladores del backend. Aquí podemos encontrar un listado con los diferentes endpoints:

ENDPOINT PRINCIPALES DE LA API	
AuthController ("/api/auth")	
POST	/register
POST	/login
ClientController ("/api/client")	
GET	/profile
PUT	/profile
GET	/dashboard
GET	/workouts
POST	/workouts
POST	/workouts/batch
PUT	/workouts/{id}
DELETE	/workouts/{id}
POST	/health/steps
POST	/health/sleep
POST	/link-trainer/{code}
GET	/notifications/sse
TrainerController ("/api/trainer")	
GET	/api/trainer/dashboard
GET	/clients
POST	/assignClient/{clientId}
GET	/client/{clientId}/progress
GET	/client/{clientId}/workouts

POST	/client/{clientId}/workouts
GET	/client/{clientId}/health
PUT	/client/{clientId}/feedback
IAController ("/api/trainer"). Mapeado en la ruta base del entrenador	
POST	/client/{clientId}/ai-chat
POST	/chat-general

4.4. Diseño de las interfaces (UI/UX)

El diseño de GymTrack se fundamenta en una estética moderna, inmersiva y orientada al rendimiento, utilizando un esquema de color oscuro por defecto. Este enfoque no solo reduce la fatiga visual sino que también transmite una identidad de marca premium dentro del sector tecnológico.

La interfaz está construida sobre la base de Material Design Components, garantizando una experiencia táctil y fluida.

Paleta de colores

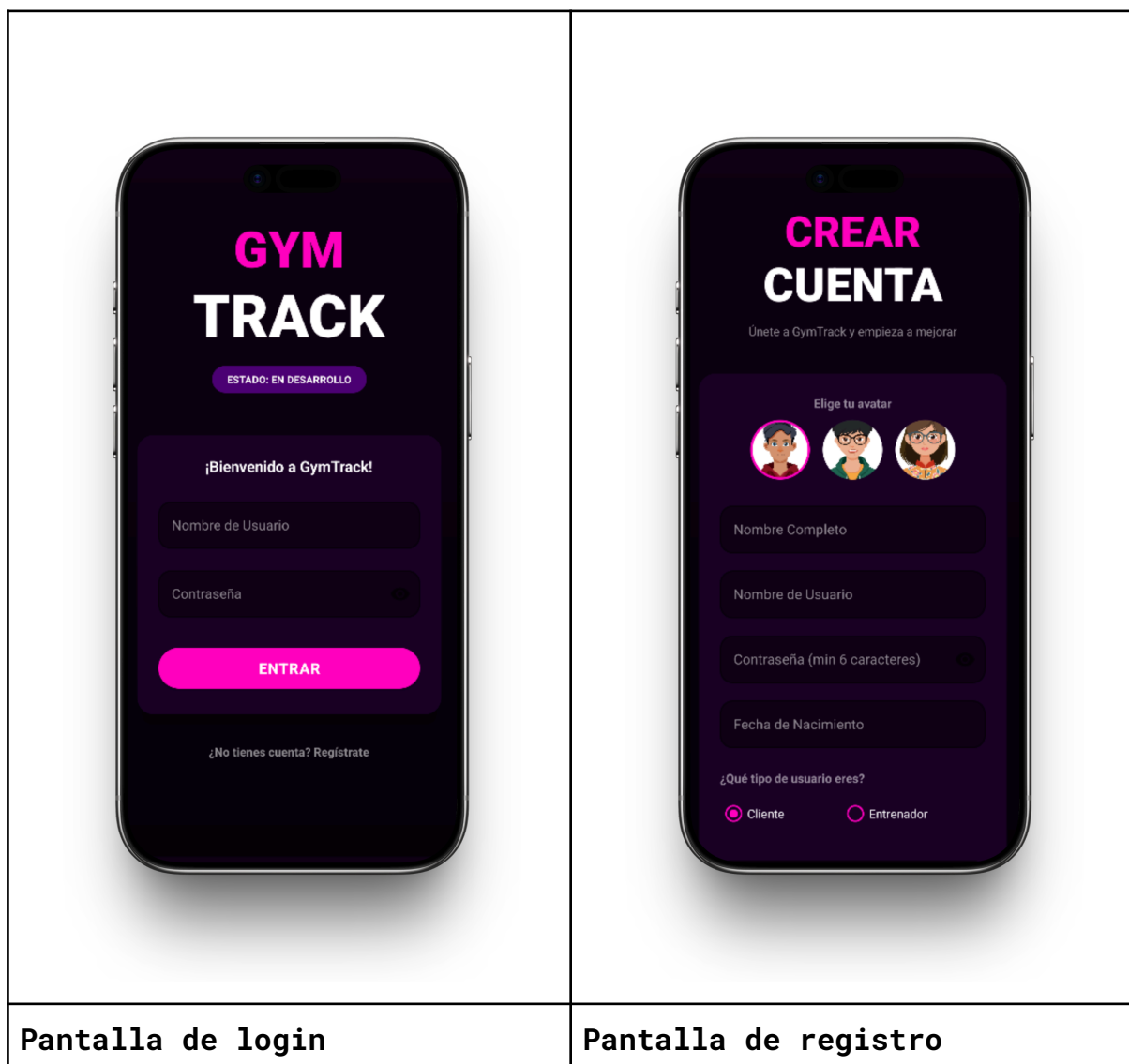
- Fondos (backgrounds): se utiliza púrpura casi negro como fondo principal de la aplicación (#0F0014).
- Color principal: se usa un tono magenta (#4D0073).
- Color secundario: se usa un tono púrpura intermedio (#4D0073).
- Tipografía: el texto principal se presenta en color blanco puro (FFFFFF) para máxima legibilidad, mientras que otros textos muestran otros colores como verde y rojo.

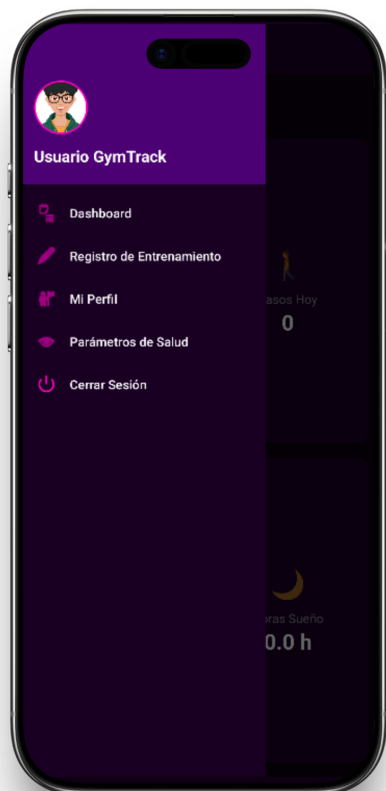
Componentes

- Botones primarios: presentan bordes redondeados, normalmente 30dp de radio con fondo magenta y texto blanco negrita.
- Campos de entrada: se usa el estilo Outlined Box de Material Design.

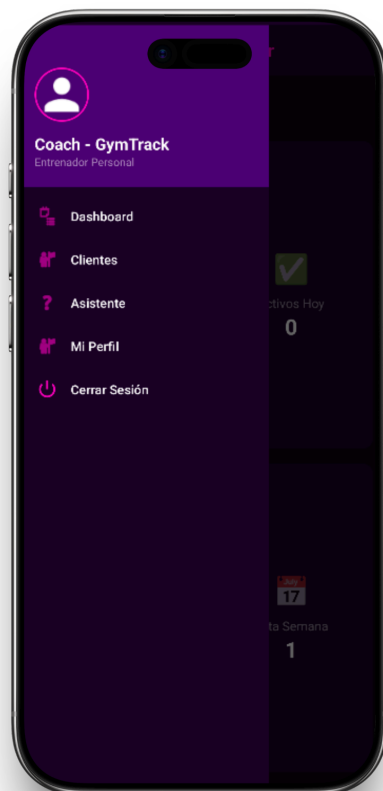
- Diálogos y modales: he usado como fondo púrpura oscuro y esquinas un poco menos redondeadas, de 20dp de radio.
- CardViews: hacemos uso de tarjetas con bordes muy redondeados de manera que los componentes queden organizados de forma coherente y visualmente legible.

En conjunto, el diseño visual prioriza la legibilidad y experiencia de usuario (UX). A continuación se muestran unos fragmentos donde se puede visualizar de forma intuitiva la interfaz general de la aplicación.

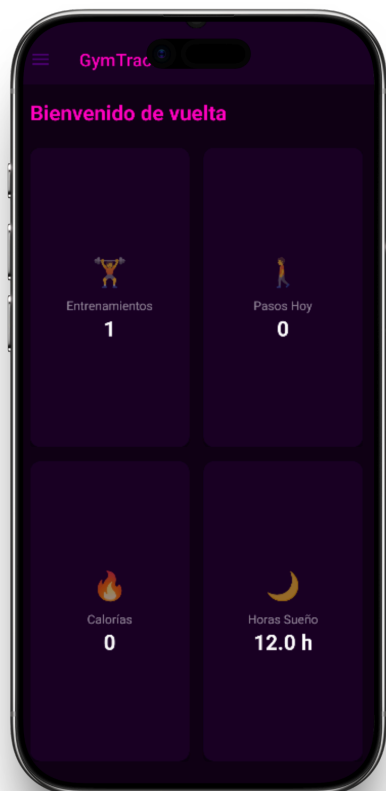




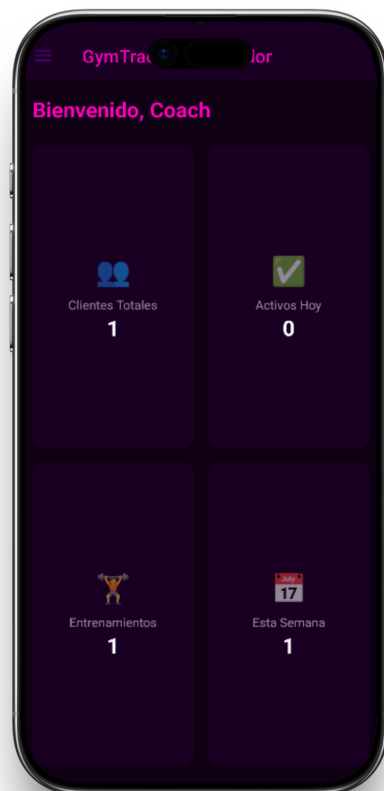
Menú clientes



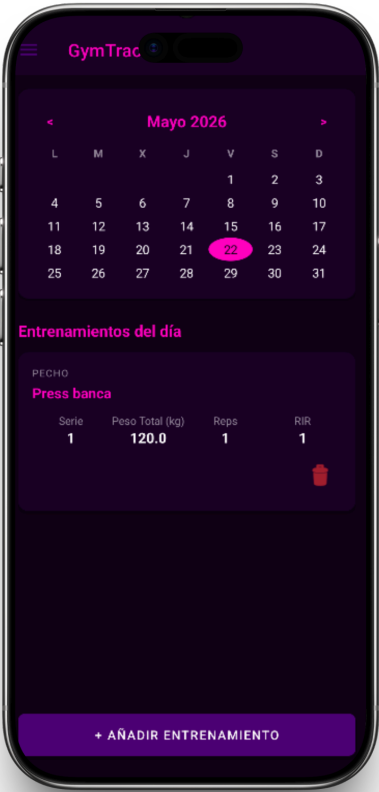
Menú entrenadores



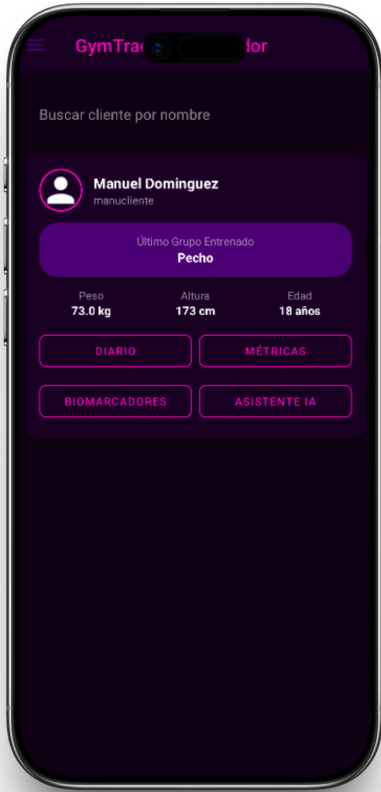
Dashboard clientes



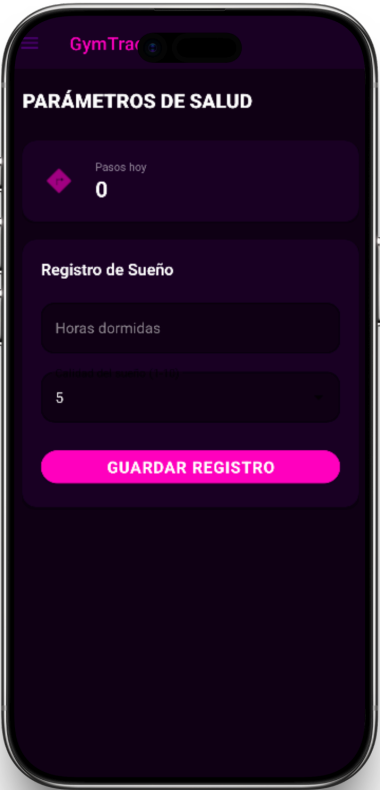
Dashboard entrenadores



Registro entrenamientos cliente



Listado de clientes



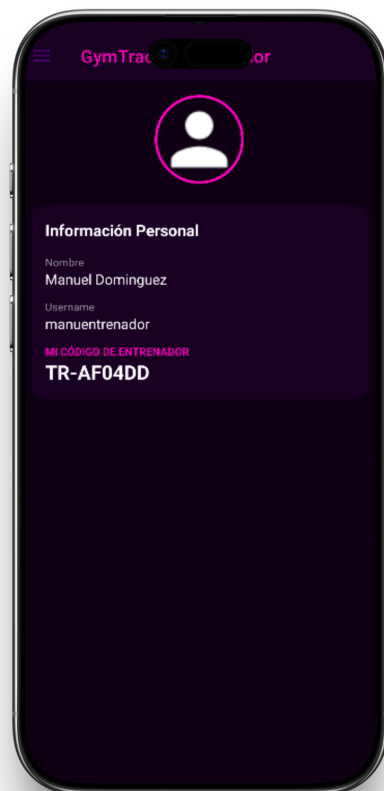
Parametros salud cliente



Asistente de IA Generativa



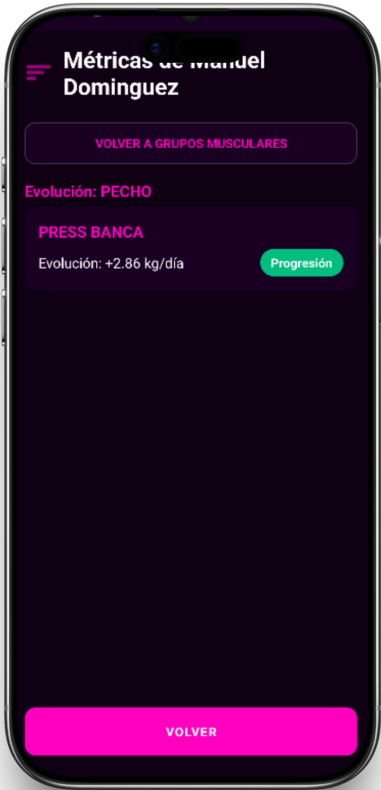
Perfil cliente



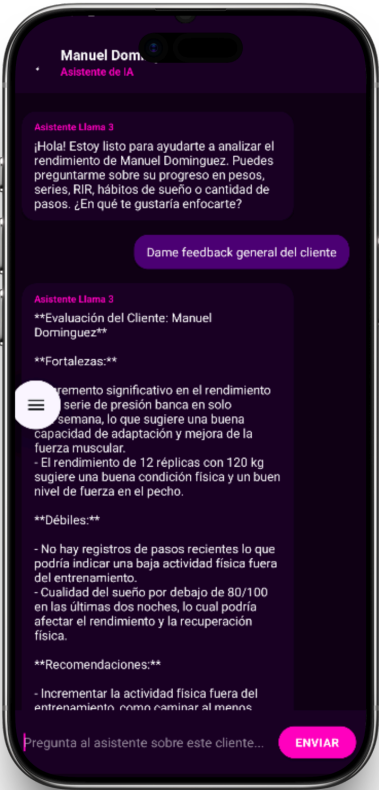
Perfil entrenador



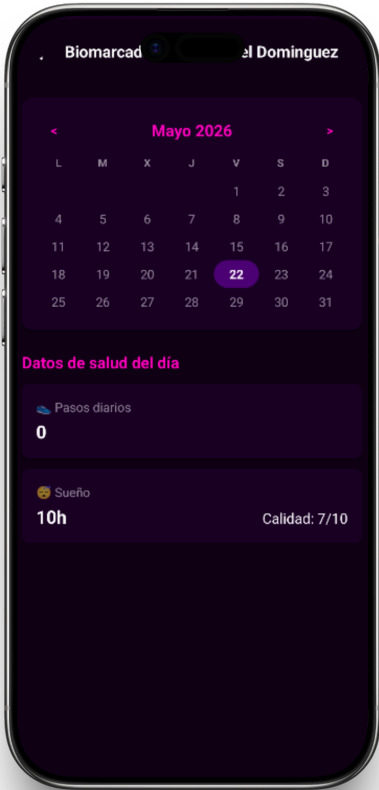
Diario del cliente



Métricas de cliente



Asistente personalizado



Biomarcadores

5. IMPLEMENTACIÓN Y ENTORNOS DE DESARROLLO

5.1. Justificación del stack tecnológico.

La selección tanto de lenguajes de programación, frameworks y herramientas empleadas para el diseño e implementación de este proyecto no ha sido arbitraria, sino que corresponde a un análisis detallado de las necesidades arquitectónicas del sistema.

El criterio principal y eje vertebrador para la configuración de este stack tecnológico ha sido la apuesta firme por el software libre y de código abierto (Open Source).

Además del modelo de licenciamiento abierto, la elección de estas tecnologías se justifica mediante los siguientes argumentos técnicos:

- Madurez y estandarización en el sector. Se han seleccionado tecnologías altamente demandadas y testadas en entornos profesionales, laborales y empresariales. Esto dota al proyecto de un enfoque que considero que es altamente profesional y realista, acercándose a los estándares de la industria.
- Alta compatibilidad e integración. Las tecnologías conforman un ecosistema cohesionado. La integración nativa entre Java, el ecosistema Spring, Hibernate y MySQL minimiza los conflictos de configuración y optimiza el flujo de datos entre la base de datos y la API REST que consume la aplicación móvil.
- Escalabilidad y mantenibilidad: el uso de herramientas modernas, junto con la contenedorización, permite que el proyecto no se conciba como una aplicación cerrado, sino como un sistema escalable el cuál sea fácil de mantener y preparar para crecer o añadir nuevas funcionalidades.

A continuación se detallan de forma más específica cada una de las tecnologías que componen la arquitectura del proyecto.

5.2. Tecnologías empleadas

Lenguajes de Programación (Java)

Para el diseño y la implementación de este proyecto, se han usado los siguientes lenguajes de programación/lenguajes de marcado:

- Java: es el lenguaje de programación principal del proyecto. Ha sido seleccionado por su robustez, fiabilidad, eficiencia y por ser multiplataforma. Construye los pilares fundamentales del desarrollo de este proyecto, especialmente la lógica de negocio del back-end.
- XML: es el lenguaje de marcado que se ha utilizado para construir las interfaces gráficas. El motor de Android Studio reconoce este lenguaje a través de sus correspondientes etiquetas para estructurar los diferentes elementos visuales de la interfaz gráfica.

Entornos de desarrollo (Android Studio y Eclipse)

La arquitectura del proyecto se divide en dos bloques principales, habiendo seleccionado el entorno de desarrollo que he considerado más adecuado para cada uno de ellos:

- Android Studio: se ha utilizado para construir el front-end en la aplicación. Su utilización se ha centrado principalmente en el diseño de las interfaces gráficas y la implementación del cliente HTTP para consumir la API REST, delegando la lógica de negocio más compleja al lado del servidor.
- Eclipse: entorno de desarrollo el cual se ha utilizado para construir todo el back-end de la aplicación. En este IDE se ha programado la API REST que recibe y gestiona las peticiones enviadas desde la aplicación móvil (el cliente), procesando toda la información y centralizando la lógica de negocio.

Frameworks (Spring Boot)

Para garantizar tanto la seguridad como la escalabilidad de la aplicación, se ha utilizado el ecosistema de Spring Boot para la construcción de todo el back-end (lógica de negocio y acceso a los datos). Se han utilizado las siguientes dependencias:

- Java Security y JWT (JSON Web Tokens): herramientas implementadas para establecer un sistema robusto de autenticación y autorización. Garantiza la protección de los endpoints de la API. Utiliza algoritmos de cifrado RSA para el manejo de las contraseñas.
- Hibernate: es un framework ORM (Object-Relational Mapping) utilizado para gestionar la capa de persistencia de datos. Permite la interacción y construcción de una base de datos relacional mapeando directamente los modelos de clases en Java a tablas mediante el uso de anotaciones en el código.

Base de datos (MySQL)

Para el almacenamiento persistente y estructurado de la información de la aplicación, se ha implementado un Sistema de Gestión de Bases de Datos Relacionales (RDBMS):

- MySQL: es el sistema que he elegido como motor de base de datos debido a su alta fiabilidad y rendimiento. Los motivos principales para la elección de este motor han sido su naturaleza de software libre (Open Source), su madurez en el mercado y su excelente compatibilidad con Java.

Control de versiones (Git y GitHub)

Se han utilizado las siguientes herramientas:

- Git: usado como sistema de control de versiones distribuido.
- GitHub: como plataforma de alojamiento del repositorio.

Para garantizar la integridad de los datos y la estabilidad de la aplicación durante su ciclo de vida, se ha utilizado una estrategia basada en el trabajo sobre diferentes ramas, permitiendo el desarrollo de nuevas funcionalidades de forma aislada sin comprometer el código principal.

Contenedorización y despliegue (Docker y Railway)

Para estandarizar el entorno de desarrollo, es imprescindible aislar los servicios. Para ello se ha utilizado la tecnología de contenedores. Esto permite que el back-end sea completamente independiente del sistema operativo o dispositivo en el cual se ejecute.

- Docker: se ha utilizado esta plataforma para encapsular tanto la base de datos como la aplicación del servidor (Spring Boot) en contenedores ligeros.
- Docker Compose: se ha empleado como herramienta complementaria para la orquestación local de contenedores.

El despliegue del backend se ha realizado mediante la plataforma Railway. Esto permite que la aplicación sea completamente funcional sin depender de un servidor en ejecución local, posibilitando que el sistema opere de manera autónoma, continua y accesible desde la nube.

Otras tecnologías

- Linux: sistema operativo usado durante todo el desarrollo. Concretamente Pop-OS, distribución sencilla basada en Ubuntu.
- Bruno: herramienta utilizada para gestionar y probar peticiones HTTP, con soporte integrado para flujos de autenticación y autorización.
- Llama3: inteligencia artificial open source a la cual se realizan las llamadas.

6. SEGURIDAD

He considerado que la seguridad de la información es un pilar fundamental en el desarrollo de cualquier software. Para garantizar la confidencialidad, integridad y disponibilidad de los datos manejados por la aplicación, se ha diseñado e implementado una capa de seguridad en el back-end.

6.1. Arquitectura de seguridad con Spring Security

Para la gestión centralizada de la seguridad en la API REST, se ha integrado Spring Security. Esta herramienta intercepta todas las peticiones HTTP entrantes mediante una cadena de filtros, evaluando si la solicitud tiene los permisos necesarios antes de que alcance los controladores.

Dado que el sistema se ha diseñado como una API RESTful, Spring Security se ha configurado para actuar bajo un modelo sin estado (stateless). Esto significa que el servidor no guarda sesiones en memoria para ningún usuario.

6.2. Autenticación stateless mediante JWT (JSON Web Token)

Para resolver la autenticación en un entorno sin estado, se ha implementado el estándar JWT. El flujo es el siguiente:

1. Se genera el token. Cuando un usuario se identifica correctamente, el back-end genera un token JWT firmado digitalmente mediante un algoritmo criptográfico y una clave secreta privada la cual solo conoce el servidor.
2. Almacenamiento en el cliente. La aplicación recibe este token y lo almacena de forma segura para su uso posterior (Encrypted Shared Preferences).
3. Tránsito en cabeceras HTTP. Para cualquier trámite posterior que requiera autorización (como consultar el perfil, modificar datos, etc.), la aplicación inyecta ese JWT en la cabecera HTTP de la petición, bajo el estándar Authorization: Bearer <Token>.

4. Validación. Spring Security intercepta la petición entrante, extrae el token de la cabecera, verifica su firma criptográfica y su vigencia. Si el token es válido y no ha expirado, procesa la petición. De lo contrario, devuelve un error HTTP 401 (No Autorizado) o 403 (Prohibido).

6.3. Protección de credenciales con BCrypt.

Dado que la seguridad es crítica, bajo ningún concepto podemos almacenar las contraseñas de los usuarios en texto plano.

Para proteger las credenciales, se ha utilizado el algoritmo de hashing BCrypt. Este está diseñado para ser computacionalmente costoso e incorpora de forma nativa la técnica de salting (añadir una cadena de datos aleatorios a la contraseña antes de ser cifrada). Esto garantiza que:

- Dos contraseñas idénticas generan hashes completamente distintos.
- El sistema es altamente resistente a ataques de fuerza bruta.
- Ni siquiera los administradores con acceso directo a la base de datos puedan conocer o realizar ingeniería inversa para obtener las contraseñas originales.

7. CONCLUSIÓN

8. LICENCIA

Este proyecto, en consonancia con la filosofía de uso de tecnologías abiertas detallada en apartados anteriores, se distribuye y publica bajo la **Licencia MIT**.

Se ha seleccionado esta licencia de software libre y código abierto (*Open Source*) por ser una de las más permisivas y estandarizadas en la comunidad de desarrolladores. La licencia MIT permite a cualquier persona que obtenga una copia del

software y de sus archivos asociados utilizar, copiar, modificar, fusionar, publicar, distribuir, sublicenciar y/o vender copias del software, con la única condición y restricción de que el aviso de derechos de autor (Copyright) y el aviso de permiso se incluyan en todas las copias o partes sustanciales del mismo.

El proyecto está registrado bajo los siguientes derechos de autor: **Copyright (c) 2026 Manuel Jesús Domínguez Diéguez**